Processing ノート(記入例)

黒石商業高校「Java 言語を利用したゲームアプリケーションソフトウェアの開発」

2018 年 8 月 28 日(火)、29 日(水)、30 日(木)

1. println()関数

1.1 説明

機能: コンソールに 1 行表示する

意味: print line: プリント・ライン: 1 行表示

使い方: println(表示したい内容);

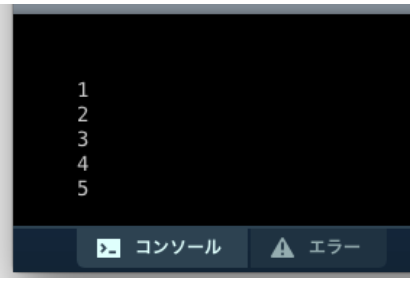
1.2 コンソールに「おはよう」と表示

```
println("おはよう");
```

1.3 コンソールに「おはよう」「こんにちは」と 1 行ずつ表示

```
println("おはよう");
println("こんにちは");
```

1.4 コンソールに1～5までの数字を1行ずつ表示



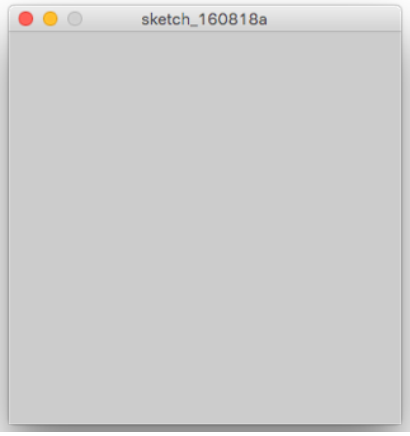

```
for (int i = 1; i <= 5; i++) {
  println(i);
}
```

2. size()関数

2.1 説明

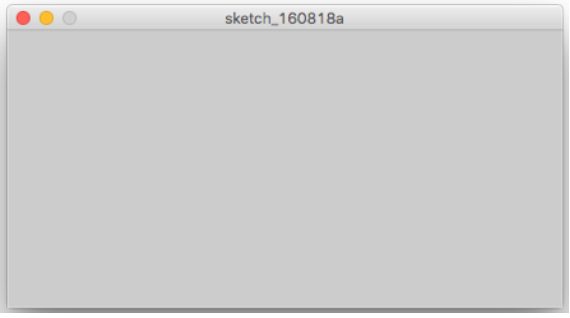
```
機能: ディスプレイ・ウィンドウの大きさを設定
意味: size: サイズ: 大きさ
使い方: size(幅, 高さ);
```

2.1 ディスプレイ・ウィンドウのサイズを320x320に




```
size(320, 320);
```

2.2 ディスプレイ・ウィンドウのサイズを 480x240 に




```
size(480, 240);
```

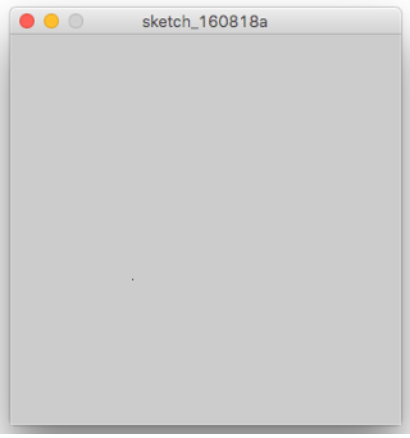
3. 描画関数

3.1 point()関数

3.1.1 説明

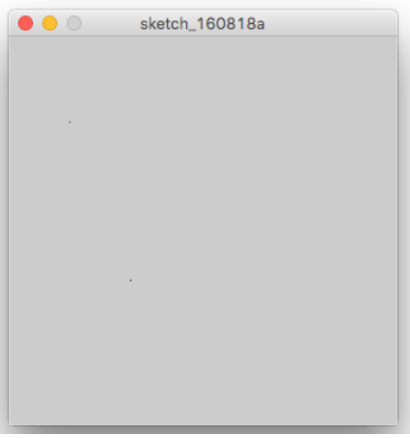
```
機能: 点を描画
意味: point: ポイント: 点
使い方: point(x, y);
```

3.1.2 点を(100, 200)に描画。ディスプレイ・ウィンドウのサイズは 320x320




```
size(320, 320);
point(100, 200);
```

3.1.3 点を(100, 200)と(50, 70)に描画。ディスプレイ・ウィンドウのサイズは320x320




```
size(320, 320);
point(100, 200);
point(50, 70);
```

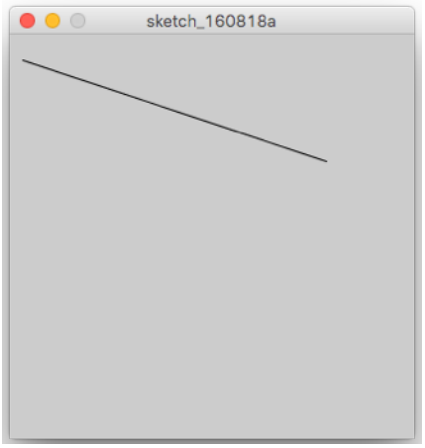
3.2 line()関数

3.2.1 説明

```
機能: 線を描画
意味: line: ライン: 線
使い方: line(x1, y1, x2, y2);
```

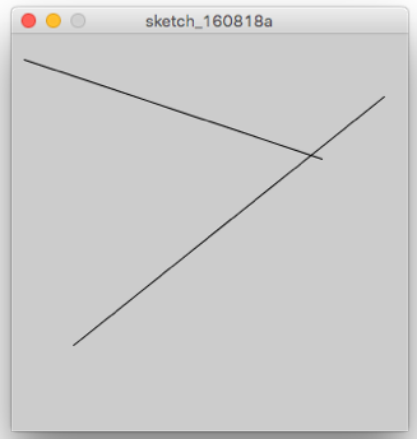
3.2.2 線を(10, 20)から(250, 100)に描画。ディスプレイ・ウィンドウのサイズは320x320。

以降、特に断らない限り、ディスプレイ・ウィンドウのサイズは 320x320 とする。




```
size(320, 320);
line(10, 20, 250, 100);
```

3.2.3 線を 2 つ描画。1 つは(10, 20)から(250, 100)、もう一つは(300, 50)から(50, 250)。



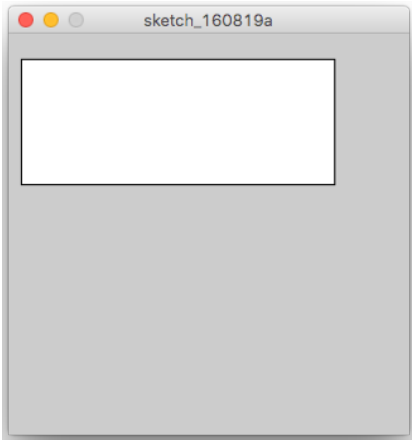

```
size(320, 320);
line(10, 20, 250, 100);
line(300, 50, 50, 250);
```

3.3 rect()関数

3.3.1 説明

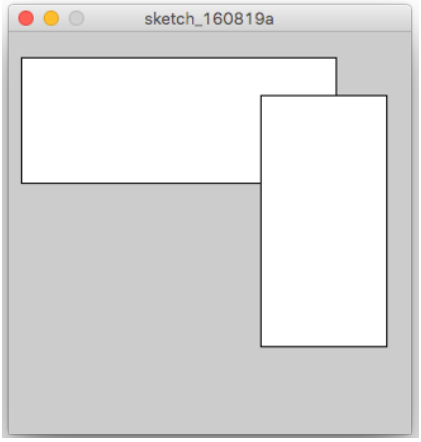
機能: 長方形を描画
意味: rectangle: レクトアングル: 長方形
使い方: rect(x, y, 幅, 高さ);

3.2.2 長方形を描画。左上が(10, 20)で、幅 250 高さ 100。



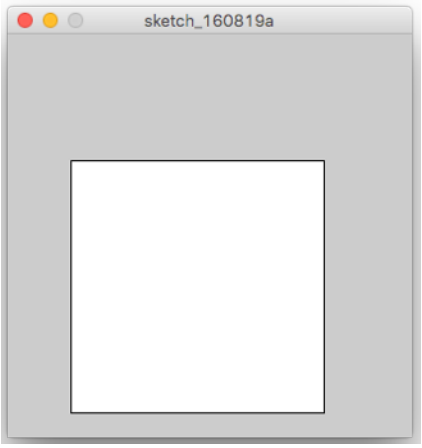

```
size(320, 320);
rect(10, 20, 250, 100);
```

3.3.3 長方形を 2 つ描画。1 つは左上が(10, 20)で、幅 250 高さ 100。もう 1 つは左上が(200, 50)で、幅 100 高さ 200。



```
size(320, 320);
rect(10, 20, 250, 100);
rect(200, 50, 100, 200);
```

3.3.4 正方形を描画。左上が(50, 100)で、1 辺の長さ 200



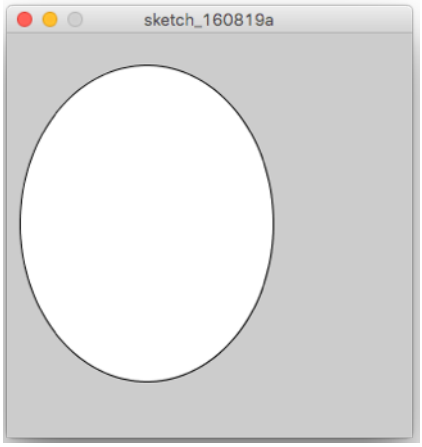

```
size(320, 320);
rect(50, 100, 200, 200);
```

3.4 ellipse()関数

3.4.1 説明

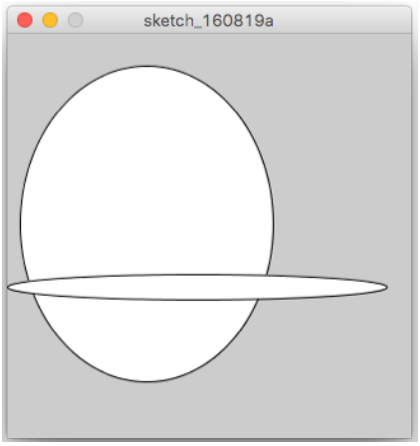
```
機能: 楕円形を描画
意味: ellipse: エリプス: 楕円形
使い方: ellipse(中心の x, 中心の y, 幅, 高さ);
```

3.4.2 楕円形を描画。中心が(110, 150)で、幅200高さ250。



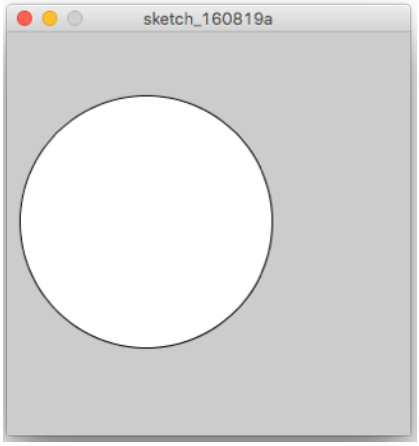

```
size(320, 320);
ellipse(110, 150, 200, 250);
```

3.4.3 楕円形を2つ描画。1つは中心が(110, 150)で、幅200高さ250。もう一つは中心が(150, 200)で、幅300高さ20。




```
size(320, 320);
ellipse(110, 150, 200, 250);
ellipse(150, 200, 300, 20);
```

3.4.4 円を描画。中心が(110, 150)で、直径が200。



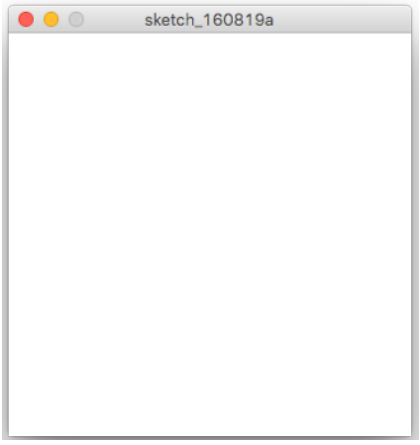

```
size(320, 320);
ellipse(110, 150, 200, 200);
```

3.5 background()関数

3.5.1 説明

```
機能: 背景を描画
意味: background: バックグランド: 背景
使い方: background(色など);
```

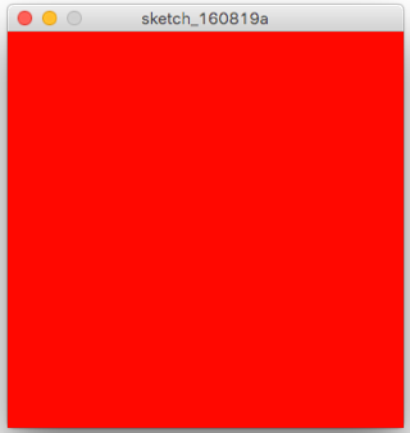
3.5.2 背景を白に。




```
size(320, 320);
```

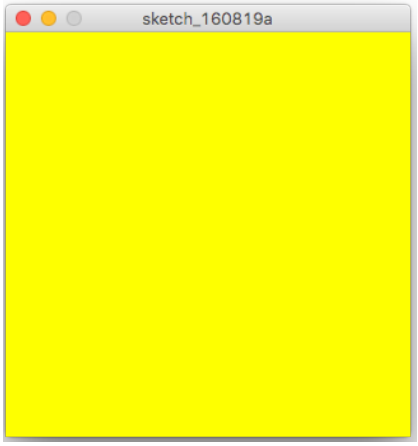
```
background(255);
```

3.5.3 背景を赤に。




```
size(320, 320);
background(255, 0, 0);
```

3.5.4 背景を黄色に。



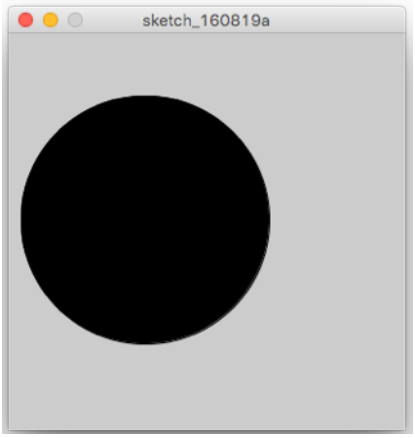

```
size(320, 320);
background(255, 255, 0);
```

3.6 fill()関数

3.6.1 説明

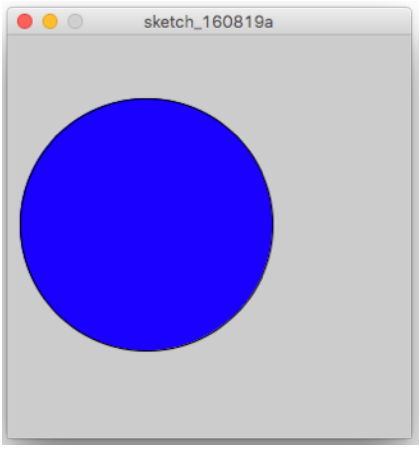
機能: 塗りの色を指定
意味: fill: フィル: 塗り
使い方: fill(色);

3.6.2 円を描画。中心が(110, 150)で、直径が200、塗りが黒。




```
size(320, 320);
fill(0);
ellipse(110, 150, 200, 200);
```

3.6.3 円を描画。中心が(110, 150)で、直径が200、塗りが青。



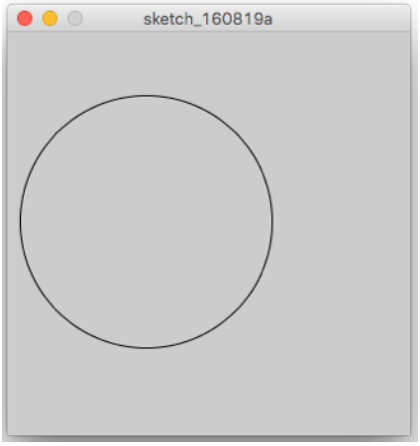
```
size(320, 320);
fill(0, 0. 255);
ellipse(110, 150, 200, 200);
```

3.7 noFill()関数

3.7.1 説明

```
機能: 色を塗らない
使い方: noFill();
```

3.7.2 円を描画。中心が(110, 150)で、直径が200、塗らない。



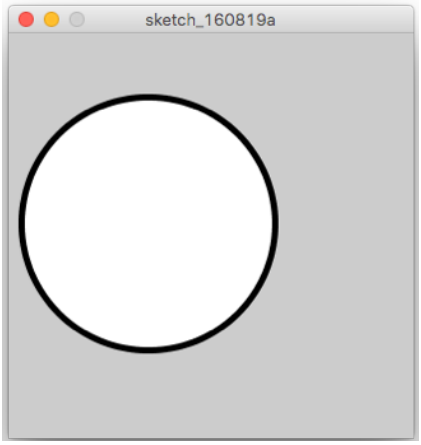

```
size(320, 320);
noFill();
ellipse(110, 150, 200, 200);
```

3.8 strokeWeight()関数

3.8.1 説明

```
機能: 線の太さを指定
意味: stroke weight: ストローク・ウェイト: 線の太さ
使い方: strokeWeight(太さ);
```

3.8.2 円を描画。中心が(110, 150)で、直径が200、線の太さ5。



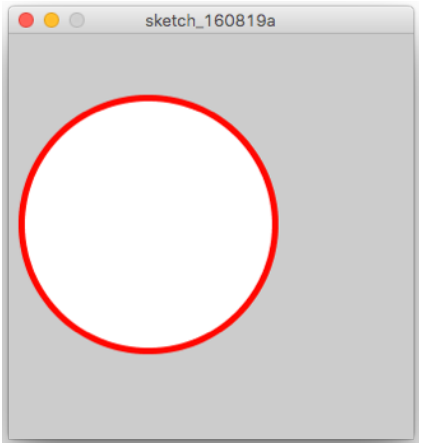

```
size(320, 320);
strokeWeight(5);
ellipse(110, 150, 200, 200);
```

3.9 stroke()関数

3.9.1 説明

```
機能: 線の色を指定
使い方: stroke(色);
```

3.9.2 円を描画。中心が(110, 150)で、直径が200、線の色が赤、線の太さ5。



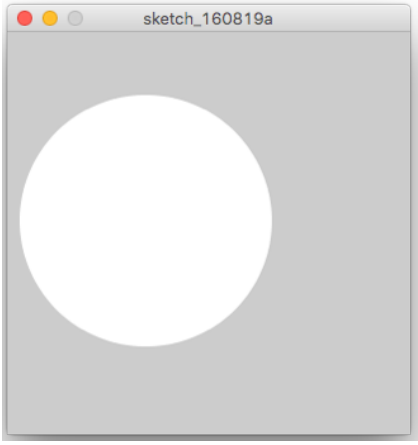
```
size(320, 320);
stroke(255, 0, 0);
strokeWeight(5);
ellipse(110, 150, 200, 200);
```

3.10 noStroke()関数

3.10.1 説明

```
機能: 線を描かない
使い方: noStroke();
```

3.10.2 円を描画。中心が(110, 150)で、直径が200、線を描かない。



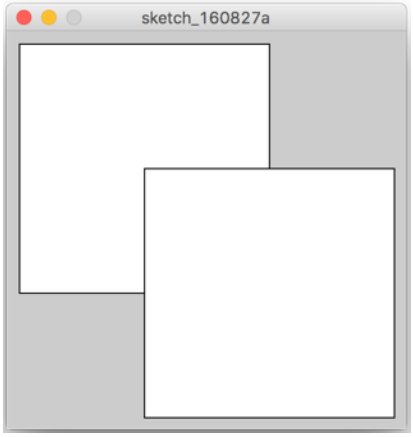

```
size(320, 320);
noStroke();
ellipse(110, 150, 200, 200);
```

3.11 順次

3.11.1 説明

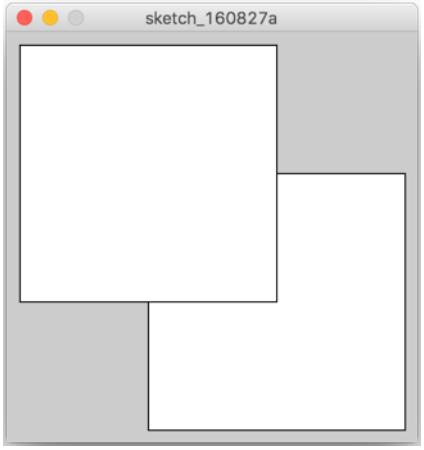
```
プログラムは書いた順番に実行される。
```

3.11.2 正方形を 2 つ描画。1 つめは左上が(10, 10)で、1 辺 200。2 つめは左上が(110, 110)で、1 辺 200。




```
size(320, 320);
rect(10, 10, 200, 200);
rect(110, 110, 200, 200);
```

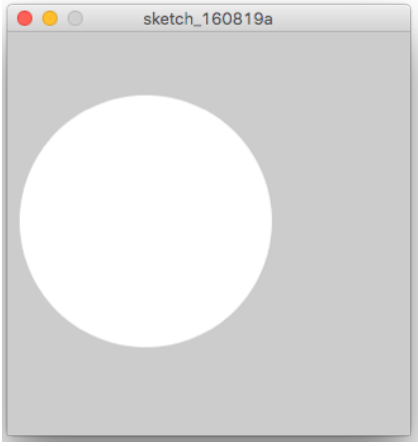
3.11.3 正方形を 2 つ描画。1 つめは左上が(110, 110)で、1 辺 200。2 つめは左上が(10, 10)で、1 辺 200。




```
size(320, 320);
rect(110, 110, 200, 200);
```

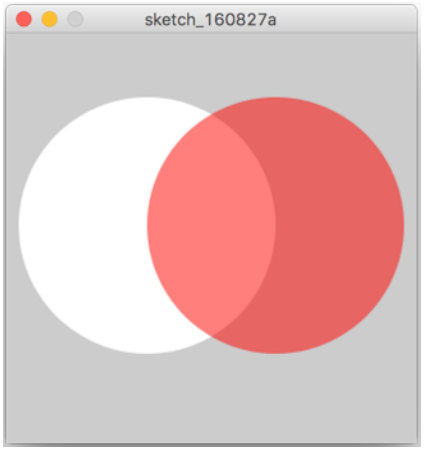
```
rect(10, 10, 200, 200);
```

3.11.4 円を描画。中心が(110, 150)で、直径が200、線を描かない。




```
size(320, 320);
noStroke();
ellipse(110, 150, 200, 200);
```

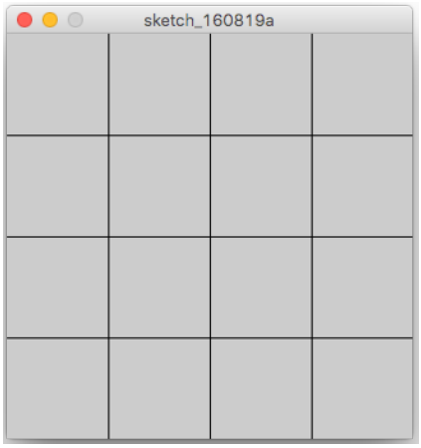
3.11.5 円を2つ描画。1つめは中心が(110, 150)で、直径が200、線を描かない、塗りが白(デフォルト)。2つめは中心が(210, 150)で、直径が200、線を描かない、塗りが赤で半透明。




```
size(320, 320);
```

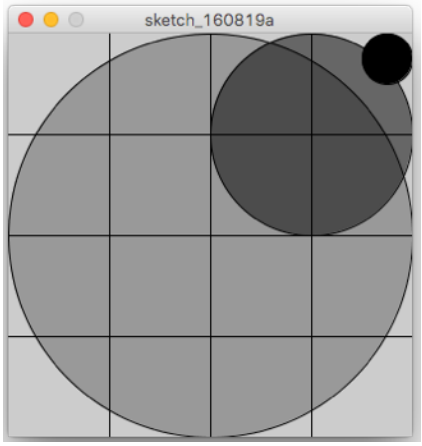
```
noStroke();
ellipse(110, 150, 200, 200);
fill(255, 0, 0, 127);
ellipse(210, 150, 200, 200);
```

3.11.6 次のように描画させる。




```
size(320, 320);
line(80, 0, 80, 320);
line(160, 0, 160, 320);
line(240, 0, 240, 320);
line(0, 80, 320, 80);
line(0, 160, 320, 160);
line(0, 240, 320, 240);
```

3.11.7 次のように描画させる。




```
size(320, 320);
line(80, 0, 80, 320);
line(160, 0, 160, 320);
line(240, 0, 240, 320);
line(0, 80, 320, 80);
line(0, 160, 320, 160);
line(0, 240, 320, 240);
fill(0);
ellipse(300, 20, 40, 40);
fill(0, 128);
ellipse(240, 80, 160, 160);
fill(0, 64);
ellipse(160, 160, 320, 320);
```

4. 変数と型

4.1 変数の宣言と型

4.1.1 説明

```
変数の宣言
変数の型 変数名;
```

```
いろいろな型
```

```
int 整数 integer インテジャー
float 小数 floating point number フローティング・ポイント・ナンバー 浮動小数点数
double 倍精度の小数 double precision floating point number ダブル・プレシジョン・
フローティング・ポイント・ナンバー 倍精度浮動小数点数
boolean 真偽値 ブーリアン ジョージ・ブールの代数
  true 真 トゥルー、false 偽 フォールス
char 文字 character キャラクター
String 文字列 ストリング
PImage 画像 Processing image プロセシング・イメージ
PFont フォント Processing font プロセシング・フォント
```

4.1.2 整数の変数 i の宣言

```
int i;
```

4.1.3 小数の変数 x の宣言

```
float x;
```

4.2 変数への値の代入

4.2.1 説明

```
=で、右の値を左に代入。
値は上書きされる。
```

4.2.2 整数の変数 i を宣言し、10 を代入し、コンソールに表示する。

```
int i;
i = 10;
println(i);
```

4.2.3 整数の変数 i を宣言し、10 を代入し、529 を代入し、コンソールに表示する。

```
int i;
i = 10;
i = 529;
println(i);
```

4.3 変数の初期化

4.3.1 説明

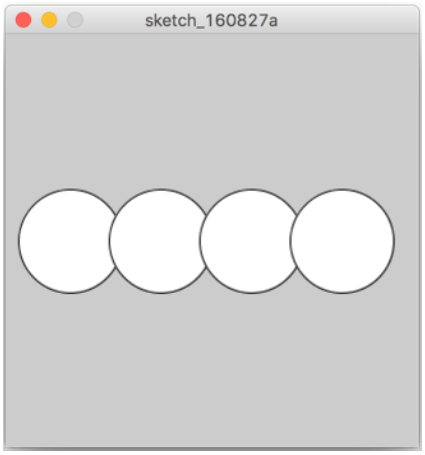
```
変数の宣言と値の代入を 1 行で行うことを変数の初期化という。
変数の型 変数名 = 値;
```

4.3.2 整数の変数 i を 10 で初期化し、コンソールに表示する。

```
int i = 10;
println(i);
```

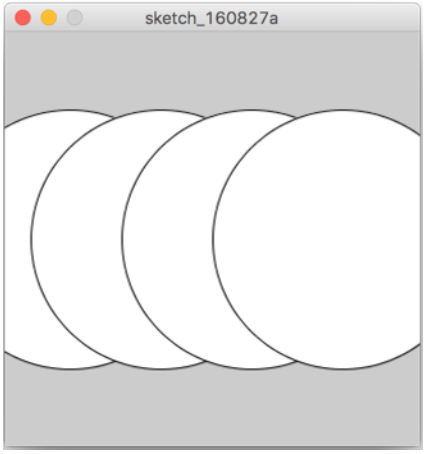
4.4 変数を活用した描画

4.4.1 円を 4 つ描く。直径はすべて 80。中心(50、160)、(120、160)、(190、160)、(260、160)。



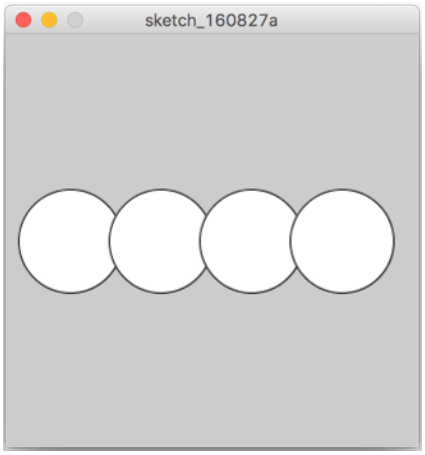
```
size(320, 320);
ellipse(50, 160, 80, 80);
ellipse(120, 160, 80, 80);
ellipse(190, 160, 80, 80);
ellipse(260, 160, 80, 80);
```

4.4.2 円を4つ描く。直径はすべて200。中心(50、160)、(120、160)、(190、160)、(260、160)。



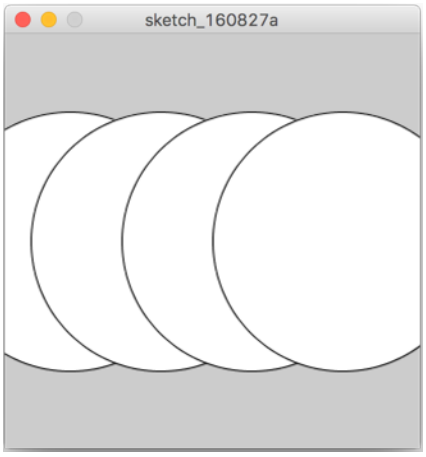

```
size(320, 320);
ellipse(50, 160, 200, 200);
ellipse(120, 160, 200, 200);
ellipse(190, 160, 200, 200);
ellipse(260, 160, 200, 200);
```

4.4.3 円を4つ描く。直径には変数dを指定し、その値は80。中心(50、160)、(120、160)、(190、160)、(260、160)。




```
size(320, 320);
int d = 80
ellipse(50, 160, d, d);
ellipse(120, 160, d, d);
ellipse(190, 160, d, d);
ellipse(260, 160, d, d);
```

4.4.4 円を 4 つ描く。直径には変数 d を指定し、その値は 200。中心(50、160)、(120、160)、(190、160)、(260、160)。




```
size(320, 320);
int d = 200
```

```
ellipse(50, 160, d, d);
ellipse(120, 160, d, d);
ellipse(190, 160, d, d);
ellipse(260, 160, d, d);
```

4.5 ディスプレイ・ウィンドウの変数

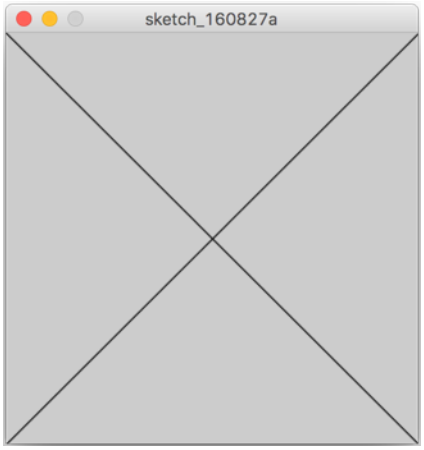
4.5.1 説明

```
Processing が用意した変数がいくつかある。
width ウィドゥス ディスプレイ・ウィンドウの幅
height ハイト ディスプレイ・ウィンドウの高さ
```

4.5.2 ディスプレイ・ウィンドウのサイズを 100x300 に設定し、ディスプレイ・ウィンドウの幅と高さをコンソールに表示する。

```
size(100, 300);
println(width);
println(height);
```

4.5.3 ディスプレイ・ウィンドウの変数を用いて以下のように描画する。




```
size(320, 320);
```

```
line(0, 0, width, height);
line(0, width, height, 0);
```

5. 演算子

5.1 さまざまな演算子

5.1.1 説明

```
= 代入
+ 足す、文字列の連結
- 引く
* 掛ける
/ 割る
% 割った余り
+= 足して代入
++ 1つ増やす
```

5.1.2 コンソールに、12 足す 3 の結果を表示する。

```
println(12 + 3);
```

5.1.3 整数の変数を i と j を用意し、それぞれ 12 と 3 を代入し、足した結果をコンソールに表示する。

```
int i = 12, j = 3;
println(i + j);
```

5.1.4 整数の変数 i を 12 で初期化する。それをコンソールに表示する。次に 3 増やし、コンソールに表示する。

```
int i = 12;
println(i);
i += 3;
```

```
println(i);
```

5.1.5 コンソールに"hello"と"world"をつなげた文字列を表示する。

```
println("hello" + "world");
```

5.2 自動型変換と演算子の優先順位

5.2.1 説明

```
整数と整数の演算→整数
小数と小数の演算→小数
整数と小数の演算→小数
何かと文字列の演算→文字列

先に演算したい内容は()でくくる
```

5.2.2 コンソールに、100 割る 3 の結果を表示する。

```
println(100 / 3);
```

5.2.3 コンソールに 100.0 割る 3.0 の結果を表示する。

```
println(100.0 / 3.0);
```

5.2.4 コンソールに 100.0 割る 3 の結果を表示する。

```
println(100.0 / 3);
```

5.2.5 以下のプログラムを実行すると、数値の部分が 123 と表示される。正しく表示するように修正せよ。

```
println("合計は" + 12 + 3 + "です。");
```

```
println("合計は" + (12 + 3) + "です。");
```

5.3 代入の制限と型変換

5.3.1 説明

```
小数の変数に、整数は代入でき、小数になる。
整数の変数に、小数を代入することはできない、データが落ちるから。

それでも代入したい場合、型変換を行う必要がある。
int(値) 値を整数に変換
float(値) 値を小数に変換
```

5.3.2 小数の変数 x を 12 で初期化する。それをコンソールに表示する。

```
float x = 12;
println(x);
```

5.3.3 次のプログラムを実行しようとするとエラーになる。型変換を利用し、エラーが出なくなるようにせよ。

```
float x = 100.0;
float y = 3;
int i = x / y;
println(i);
```

```
float x = 100.0;
float y = 3;
int i = int(x / y);
println(i);
```

6. くりかえし

6.1 for 文

6.1.1 説明

```
for（初期化; くりかえしを続ける条件; 変化）{
  くりかえしたい内容
}
```

```
条件式
値　関係演算子　値

結果は真偽値になる。
  成り立つ true / 成り立たない false
```

```
関係演算子
== 等しい
!= 等しくない
< よりも小さい
<= 以下
> よりも大きい
>= 以上
```

```
論理演算子
&& かつ
|| または
```

6.1.2 コンソールに 0 から 10 まで 1 つずつ表示せよ。

```
for (int i = 0; i <= 10; i++) {
  println(i);
}
```

6.1.3 コンソールに1から10まで1つずつ表示せよ。

```
for (int i = 1; i <= 10; i++) {
  println(i);
}
```

6.1.4 コンソールに0、1、2、…と10個の数字を表示せよ。

```
for (int i = 0; i < 10; i++) {
  println(i);
}
```

6.1.5 コンソールに7、8、9、…、13と表示せよ。

```
for (int i = 7; i <= 13; i++) {
  println(i);
}
```

6.1.6 コンソールに10、9、…、0と表示せよ。

```
for (int i = 10; i >= 0; i--) {
  println(i);
}
```

6.1.7 コンソールに1、3、5、…、19と表示せよ。

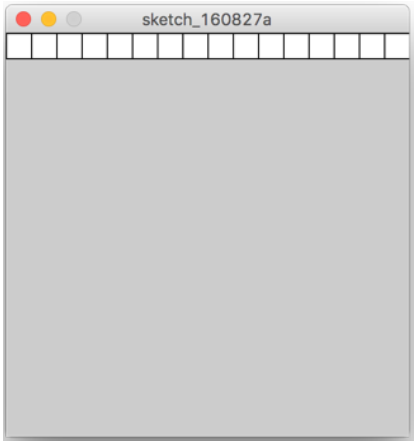
```
for (int i = 1; i <= 19; i += 2) {
  println(i);
}
```

6.1.8 コンソールに2、4、8、…、128と表示せよ。

```
for (int i = 2; i <= 128; i *= 2) {
  println(i);
}
```

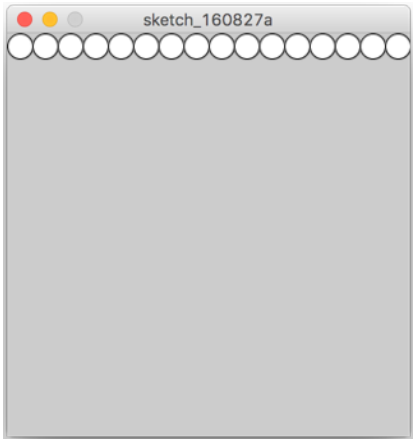
6.2 for 文による描画

6.2.1 図のように描画せよ。




```
size(320, 320);
for (int i = 0; i < width; i += 20) {
  rect(i, 0, 20, 20);
}
```

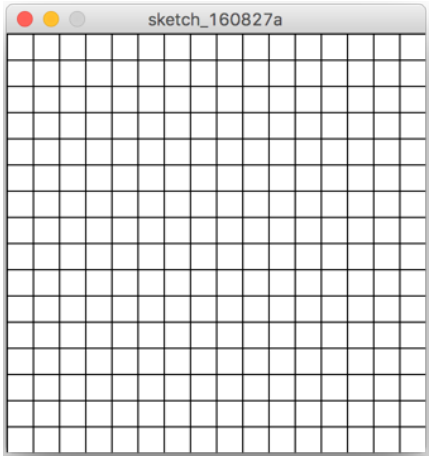
6.2.2 図のように描画せよ。



```
size(320, 320);
for (int i = 10; i < width; i += 20) {
  ellipse(i, 10, 20, 20);
}
```

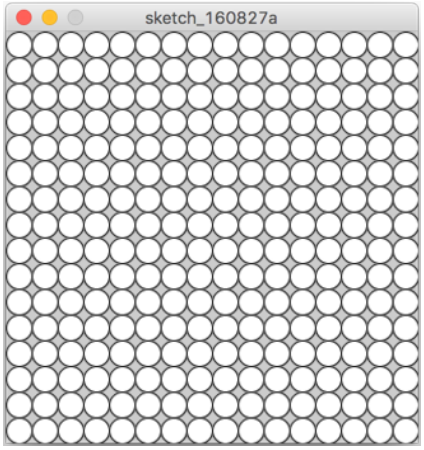
6.3 2 重ループ

6.3.1 図のように描画せよ。




```
size(320, 320);
for (int j = 0; j < height; j += 20) {
  for (int i = 0; i < width; i += 20) {
    rect(i, j, 20, 20);
  }
}
```

6.3.2 図のように描画せよ。




```
size(320, 320);
for (int j = 10; j < height; j += 20) {
  for (int i = 10; i < width; i += 20) {
    ellipse(i, j, 20, 20);
  }
}
```

7. アニメーション

7.1 説明

アニメーションはパラパラマンガのように、描画を切り替えて実現されている。
パラパラマンガの１コマ１コマのことをフレーム frame という。

アニメーションを使用する Processing のプログラムの構造
全体で使用する変数の宣言

```
void setup() {
  最初に１回だけ実行する内容
}

void draw() {
```

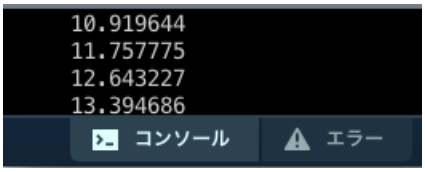
```
  毎フレーム実行する内容
}
```

7.2 フレーム・レートの設定

7.2.1 説明

```
1秒間のフレーム数をフレーム・レートという
frameRate(値); フレームレートを設定
frameRate 変数: フレーム・レートの実測値
```

7.2.2 最初にフレーム･レートを 20 に設定し､実測値をコンソールに毎フレーム表示する。




```
void setup() {
  frameRate(20);
}

void draw() {
  println(frameRate);
}
```

7.3 経過時間

7.3.1 説明

```
millis(): 経過時間を取得する。単位はミリ秒(1秒=1000ミリ秒)
```

7.3.2 経過時間(ミリ秒)をコンソールに毎フレーム表示する。

```
void setup() {
}

void draw() {
  println(millis());
}
```

7.3.3 経過時間(秒)をコンソールに毎フレーム表示する。

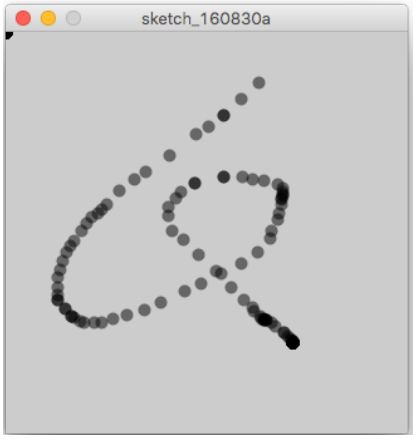
```
void setup() {
}

void draw() {
  println(millis() / 1000);
}
```

7.4 マウス変数

7.4.1 説明

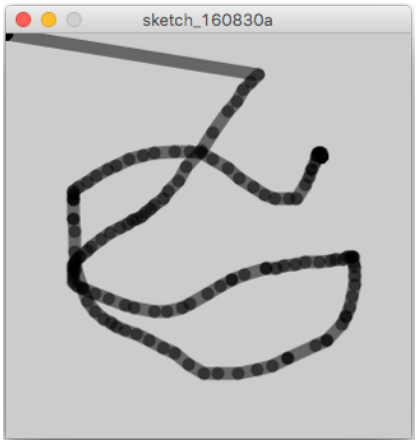
```
現在のマウスの位置 mouseX、mouseY
1フレーム前のマウスの位置 pmouseX、pmouseY
```

7.4.2 マウスの位置に点を描画する。描線の太さは 10、色は黒の半透明。




```
void setup() {
  size(320, 320);
  strokeWeight(10); // 描線の太さを 10
  stroke(0, 128); // 描線を黒の半透明
}

void draw() {
  point(mouseX, mouseY);
}
```

7.4.3 マウスの現在の位置と 1 フレーム前の位置を結ぶ線を描画する。描線の太さは 10。



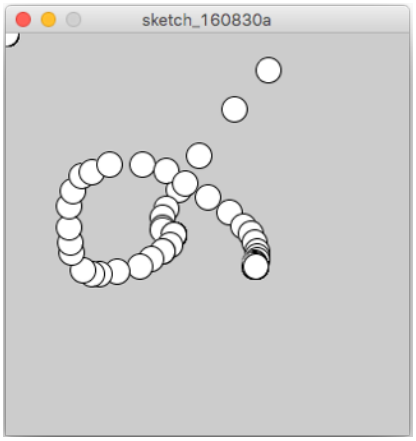
```
void setup() {
  size(320, 320);
  strokeWeight(10); // 描線の太さを 10
  stroke(0, 128); // 描線を黒の半透明
}

void draw() {
  line(mouseX, mouseY, pmouseX, pmouseY);
}
```

7.5 残像と background()関数

7.5.1 説明

```
background(色など); 背景を塗りつぶす。
draw()関数の中で使うと、残像を消すことができる。
```

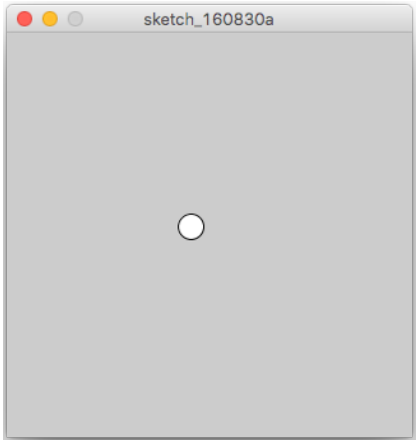
7.5.2 マウスの位置に直径 20 の円を描く。




```
void setup() {
  size(320, 320);
}

void draw() {
```

```
  ellipse(mouseX, mouseY, 20, 20);
}
```

7.5.3 マウスの位置に直径 20 の円を描く。残像が出ないようにする。




```
void setup() {
  size(320, 320);
}

void draw() {
  background(204); // 残像を消す
  ellipse(mouseX, mouseY, 20, 20);
}
```

8. 条件分岐

8.1 if 文

8.1.1 説明

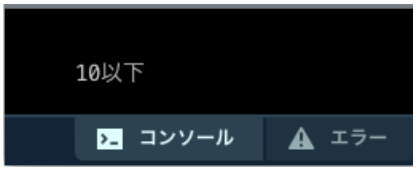
```
if (条件式) {
  条件式が成り立ったときに実行する内容
}
```

```
if (条件式) {
```

```
  条件式が成り立ったときに実行する内容
} else {
  条件式が成り立たなかったときに実行する内容
}
```

```
if (条件式 1) {
  条件式 1 が成り立ったときに実行する内容
} else if (条件式 2) {
  条件式 2 が成り立たなかったときに実行する内容
} else {
  条件式が成り立たなかったときに実行する内容
}
```

8.1.2 整数の変数 i を適当な値で初期化する。i の値が 10 以下のときに「10 以下」とコンソールに表示

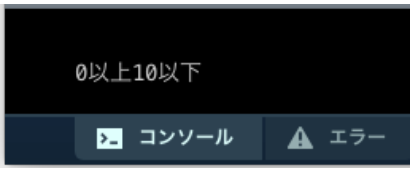



```
int i = 5;

if (i <= 10) {
  println("10 以下");
}
```

8.1.3 整数の変数 i を適当な値で初期化する。i の値が 10 以下のときに「10 以下」、そうでないときに「10 より大きい」とコンソールに表示

```
int i = 15;

if (i <= 10) {
  println("10 以下");
} else {
  println("10 より大きい");
}
```

8.1.4 整数の変数 i を適当な値で初期化する。i の値が 0 以上 10 以下のときに「0 以上 10 以下」、そうでないときに「0 より小さいか 10 より大きい」とコンソールに表示




```
int i = 5;

if (i >= 0 && i <= 10) {
  println("0 以上 10 以下");
} else {
  println("0 より小さいか 10 より大きい");
}
```

9. マウス

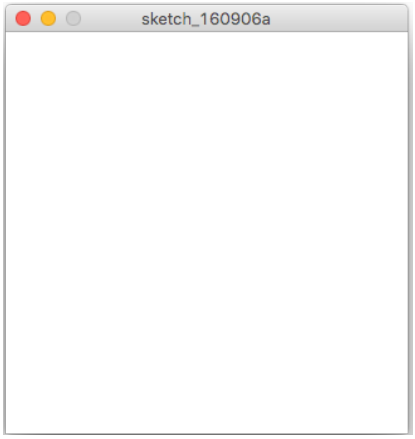
9.1 マウス変数 ※7.3 で説明しなかったもの

9.1.1 説明

```
マウスを押している mousePressed
押しているとき true、押していないとき false

最後に押したボタン mouseButton
左 LEFT、真ん中 CENTER、右 RIGHT
```

```
if (mousePressed == true)と if (mousePressed)は同じ

mousePressed の値          mousePressed == true の値
true                       true
false                      false
```

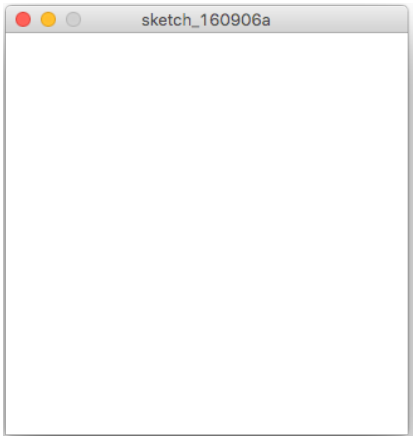
9.1.2 マウスを押したとき背景を黒に、押していないとき背景を白に




```
void setup() {
  size(320, 320);
}

void draw() {
  if (mousePressed == true) {
    background(0);
  } else {
    background(255);
  }
}
```

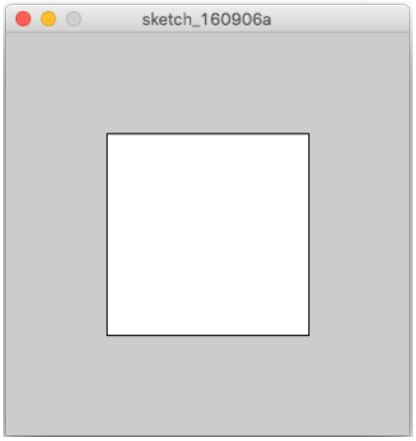
9.1.2 マウスを押したとき左ボタンなら背景を赤に、それ以外のボタンなら黒に、押してい

ないとき背景を白に



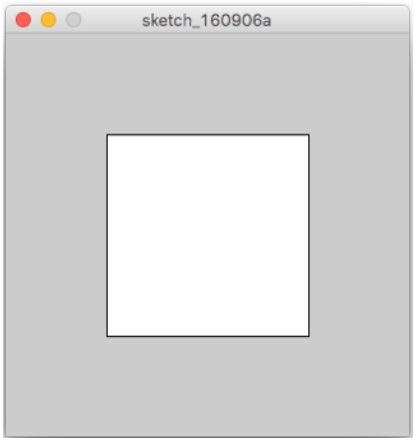

```
void setup() {
  size(320, 320);
}

void draw() {
  if (mousePressed == true) {
    if (mouseButton == LEFT) {
      background(255, 0, 0);
    } else {
      background(0);
    }
  } else {
    background(255);
  }
}
```

9.1.3 1 辺 160 の正方形を中央に描く。




```
void setup() {
  size(320, 320);
}

void draw() {
  rect(80, 80, 160, 160);
}
```

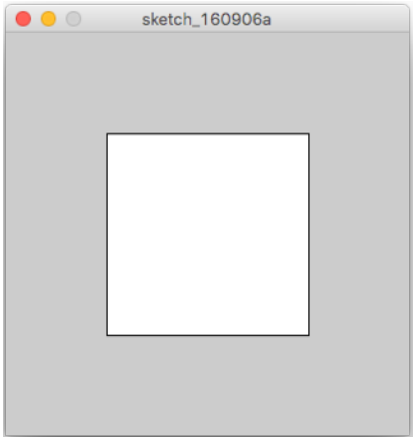
9.1.4 1 辺 160 の正方形を中央に描く。マウスの x 座標が 80 以上なら色を黒く、そうでなければ白く塗る。




```
void setup() {
```

```
  size(320, 320);
}

void draw() {
  if (mouseX >= 80) {
    fill(0);
  } else {
    fill(255);
  }
  rect(80, 80, 160, 160);
}
```

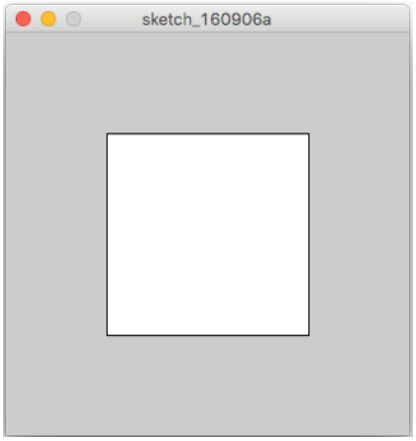
9.1.5 1 辺 160 の正方形を中央に描く。マウスの x 座標が 80 以上、y 座標が 80 以上なら色を黒く、そうでなければ白く塗る。




```
void setup() {
  size(320, 320);
}

void draw() {
  if (mouseX >= 80 && mouseY >= 80) {
    fill(0);
  } else {
```

```
    fill(255);
  }
  rect(80, 80, 160, 160);
}
```

9.1.6 1 辺 160 の正方形を中央に描く。マウスの x 座標が 80 以上 240 以下、y 座標が 80 以上 240 以下なら色を黒く、そうでなければ白く塗る。




```
void setup() {
  size(320, 320);
}

void draw() {
  if (mouseX >= 80 && mouseX <= 240 && mouseY >= 80 && mouseY <= 240) {
    fill(0);
  } else {
    fill(255);
  }
  rect(80, 80, 160, 160);
}
```

9.2 円形の当たり判定と dist 関数

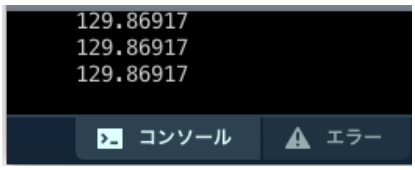
9.2.1 説明

2 点間の距離を求める

dist(点 1 の x 座標, 点 1 の y 座標, 点 2 の x 座標, 点 2 の y 座標)

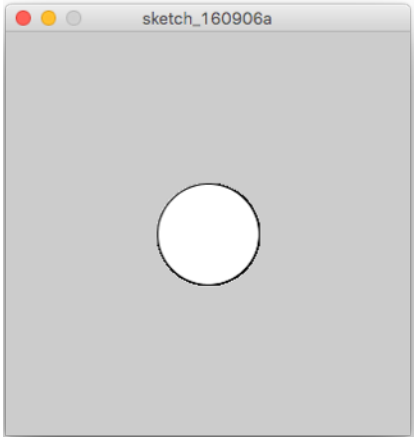
dist 関数を使うと、円形の当たり判定を行うことができる。

9.2.2 マウスと画面の中心の間の距離をコンソールに表示



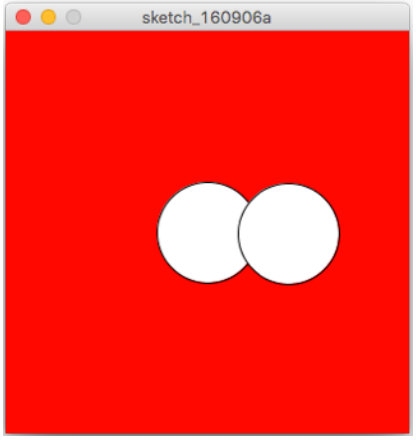

```
void setup() {
  size(320, 320);
}

void draw() {
  println(dist(mouseX, mouseY, width / 2, height / 2));
}
```

9.2.3 画面中央に半径 40(直径 80)の円を描く。マウスが円の中に入っていれば黒、そうでなければ白で円を塗る。



```
int r = 40;

void setup() {
  size(320, 320);
}

void draw() {
  if (dist(mouseX, mouseY, width / 2, height / 2) <= r) {
    fill(0);
  } else {
    fill(255);
  }
  ellipse(width / 2, height / 2, 2 * r, 2 * r);
}
```

9.2.4 画面中央とマウスを中央に半径 40(直径 80)の円をそれぞれ描く。2 つの円が交差していれば背景を赤、そうでなければ灰色にする。




```
int r = 40;

void setup() {
  size(320, 320);
```

```
}

void draw() {
  if (dist(mouseX, mouseY, width / 2, height / 2) <= r + r) {
    background(255, 0, 0);
  } else {
    background(204);
  }
  ellipse(width / 2, height / 2, 2 * r, 2 * r);
  ellipse(mouseX, mouseY, 2 * r, 2 * r);
}
```

10. キー

10.1 キー変数

10.1.1 説明

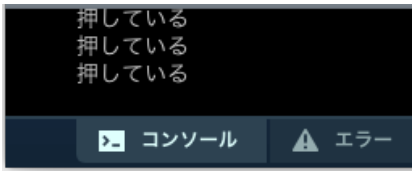
keyPressed 変数
キーを押している true、押していない false

key 変数
最後に押したキー
たとえば、「a」なら'a'
修飾キーのとき CODED

keyCode 変数
最後に押した修飾キー
UP、DOWN、LEFT、RIGHT など

10.1.2 キーを押しているとコンソールに「押している」、そうでなければ「押していない」

と表示




```
void setup() {
}

void draw() {
  if (keyPressed == true) {
    println("押している");
  } else {
    println("押していない");
  }
}
```

10.1.3 キーを押しているとコンソールにその値を、そうでなければ「押していない」と表示

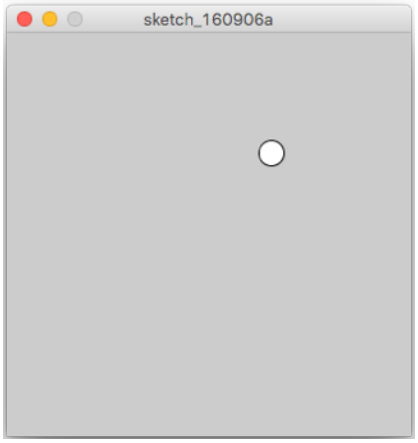
```
void setup() {
}

void draw() {
  if (keyPressed == true) {
    println(key);
  } else {
    println("押していない");
```

```
  }
}
```

10.1.4 修飾キー以外のキーを押しているとコンソールにその値を、上下キーは、それぞれ「上」「下」を、そうでなければ「押していない」と表示。

```
void setup() {
}

void draw() {
  if (keyPressed == true) {
    if (key != CODED) {
      println(key);
    } else if (keyCode == UP) {
      println("上");
    } else if (keyCode == DOWN) {
      println("下");
    }
  } else {
    println("押していない");
  }
}
```

10.1.5 画面中央に直径 20 の円を表示する。上下左右キーで、上下左右に移動させる。




```
int d = 20;
int x, y;

void setup() {
  size(320, 320);
  x = width / 2;
  y = height / 2;
}

void draw() {
  background(204);
  if (keyPressed == true && key == CODED) {
    if (keyCode == UP) {
      y--;
    } else if (keyCode == DOWN) {
      y++;
    } else if (keyCode == LEFT) {
      x--;
    } else if (keyCode == RIGHT) {
      x++;
    }
```

```
  }
  ellipse(x, y, d, d);
}
```

11. 画像

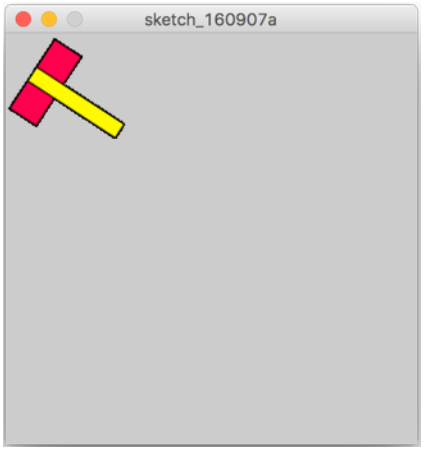
11.1 PImage クラスと image()関数

11.1.1 説明

```
Processing で使用できる画像: GIF、JPEG、PNG、SVG
```

```
画像を使うには
1. 画像を Processing スケッチにドラッグ・アンド・ドロップ
2. 画像を表す変数を宣言
  PImage 変数名;
3. 変数に画像を読み込む
  変数名 = loadImage(画像ファイル名);
4. 画像を表示する
  image(変数名, x 座標, y 座標);
```

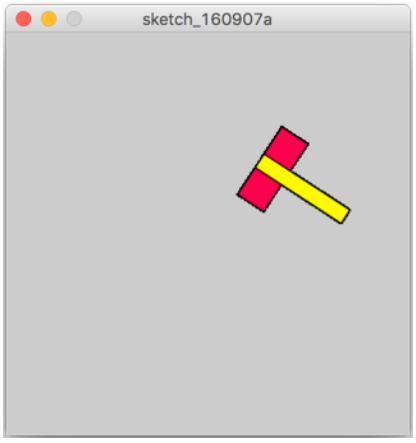
11.1.2 画像 hammer.png を表示せよ。具体的には、画像を表す変数 hammer を用意し、画像を読み込み、画像の左上の座標を(0, 0)にして表示すること。




```
PImage hammer;
```

```
void setup() {
  size(320, 320);

  hammer = loadImage("hammer.png");
}

void draw() {
  image(hammer, 0, 0);
}
```

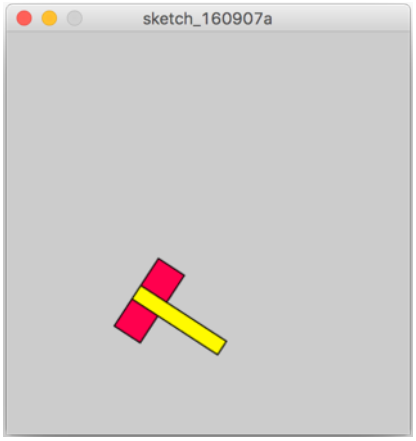
11.1.3 画像 hammer.png を、画像の左上の座標を(180, 70)にして表示




```
PImage hammer;

void setup() {
  size(320, 320);

  hammer = loadImage("hammer.png");
}

void draw() {
  image(hammer, 180, 70);
```

```
}
```

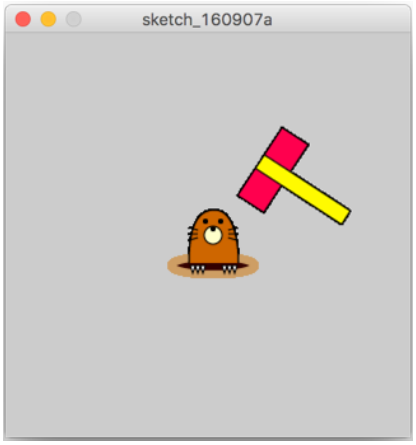
11.1.4 画像 hammer.png を、画像の左上の座標をマウスの中心にして表示。残像は消すこと。




```
PImage hammer;

void setup() {
  size(320, 320);

  hammer = loadImage("hammer.png");
}

void draw() {
  background(204);

  image(hammer, mouseX, mouseY);
}
```

11.1.5 画像 hammer.png を、画像の左上の座標を(180, 70)にして表示。画像 mole.png を、

画像の左上の座標を(128, 128)にして表示。




```
PImage hammer;
PImage mole;

void setup() {
  size(320, 320);

  hammer = loadImage("hammer.png");
  mole = loadImage("mole.png");
}

void draw() {
  image(hammer, 180, 70);
  image(mole, 128, 128);
}
```

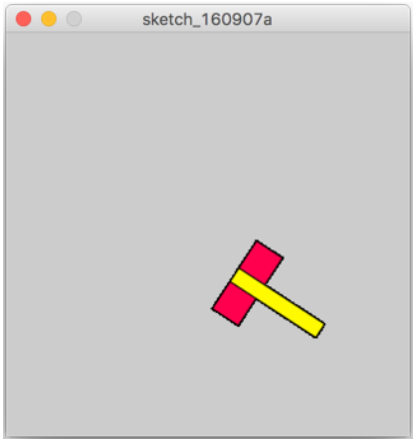
11.2 imageMode()関数

11.2.1 説明

```
画像を表示するときの座標の指定方法を設定
imageMode(モード);
```

```
モード: CORNER 画像の左上(デフォルト)、CENTER 画像の中心、ほか
```

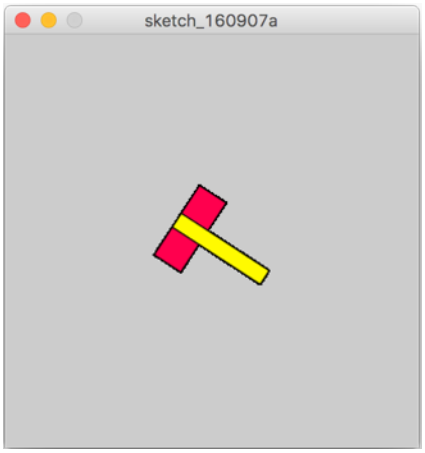
11.2.2 画像 hammer.png を、画像の左上の座標をディスプレイ・ウィンドウの中心にして表示




```
PImage hammer;

void setup() {
  size(320, 320);

  hammer = loadImage("hammer.png");
}

void draw() {
  image(hammer, width / 2, height / 2);
}
```

11.2.3 画像 hammer.png を、画像の中心の座標をディスプレイ・ウィンドウの中心にして

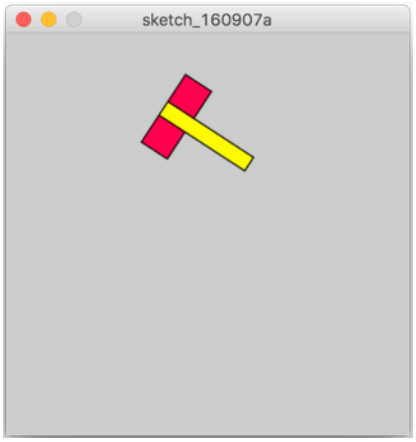
表示




```
PImage hammer;

void setup() {
  size(320, 320);

  hammer = loadImage("hammer.png");
  imageMode(CENTER);
}

void draw() {
  image(hammer, width / 2, height / 2);
}
```

11.2.4 画像 hammer.png を、画像の中心の座標をマウスの中心にして表示。残像は消すこ

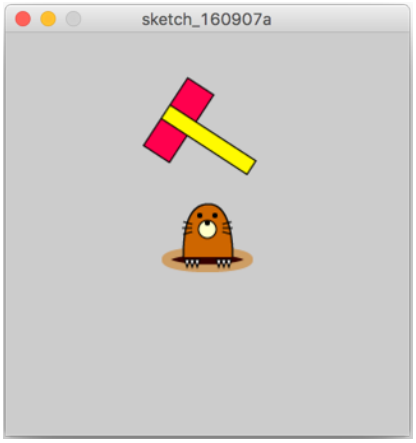
と。




```
PImage hammer;

void setup() {
  size(320, 320);

  hammer = loadImage("hammer.png");
  imageMode(CENTER);
}

void draw() {
  background(204);

  image(hammer, mouseX, mouseY);
}
```

11.2.5 画像 hammer.png を、画像の中心の座標をマウスの中心にして表示。画像 mole.png

を、ディスプレイ・ウィンドウの中心に表示。残像は消すこと。




```
PImage hammer;
PImage mole;

void setup() {
  size(320, 320);

  hammer = loadImage("hammer.png");
  mole = loadImage("mole.png");

  imageMode(CENTER);
}

void draw() {
  background(204);

  image(hammer, mouseX, mouseY);
  image(mole, width / 2, height / 2);
}
```

12. フォントと文字列の表示

12.1 PFont クラスと text()関数

12.1.1 説明

文字列を表示するには

1. フォントを表す変数を宣言

   PFont 変数名;

2. 変数にフォントを生成して読み込む

   変数名 = createFont(フォント名, サイズ);

3. フォントを設定する

   textFont(変数名);

4. 描画色を設定する

   fill(色);

5. 文字列を表示する

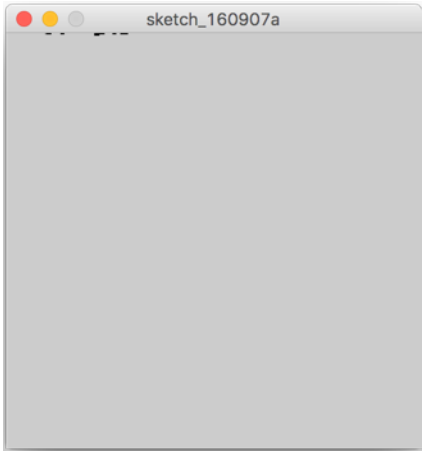
   text(文字列, x 座標, y 座標);

12.1.2 サイズが 20 ポイントの「MS ゴシック」フォントを使い、「こんにちは」と座標(20, 20)に表示

```
PFont font;


void setup() {
  size(320, 320);
```

```
  font = createFont("MS Gothic", 20);
  textFont(font);
  fill(0);
}

void draw() {
  text("こんにちは", 20, 20);
}
```

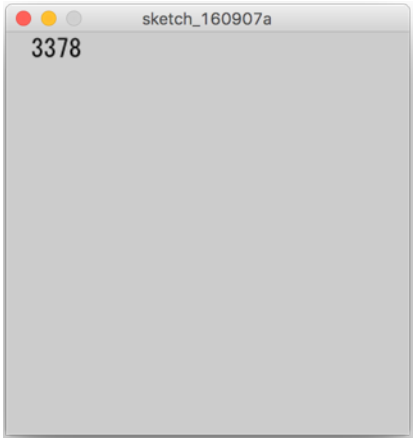
12.1.3 サイズが 20 ポイントの「MS ゴシック」フォントを使い、「こんにちは」と座標(0, 0)に表示




```
PFont font;

void setup() {
  size(320, 320);

  font = createFont("MS Gothic", 20);
  textFont(font);
  fill(0);
}
```

```
void draw() {
  text("こんにちは", 0, 0);
}
```

12.1.4 サイズが 20 ポイントの「MS ゴシック」フォントを使い、アプリが実行を開始してからの時間(単位はミリ秒)を座標(20, 20)に表示。残像は消すこと。



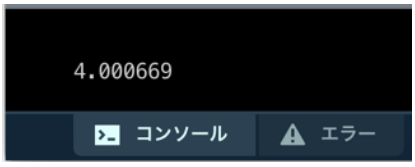

```
PFont font;

void setup() {
  size(320, 320);

  font = createFont("MS Gothic", 20);
  textFont(font);
  fill(0);
}

void draw() {
  background(204);

  text(millis(), 20, 20);
}
```

13. 乱数

13.1 説明

```
random(値)　0 から値の間の乱数を作る(値は含まない)
random(最小値, 最大値)　最小値から最大値の間の乱数を作る(最大値は含まない)
```

13.2 0 から 10 の間の乱数をコンソールに表示(10 は含まない)

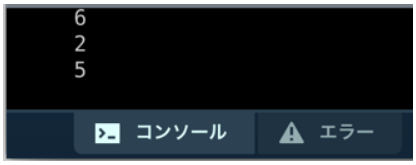



```
println(random(10));
```

13.3 サイコロをふって出た目の数を表示

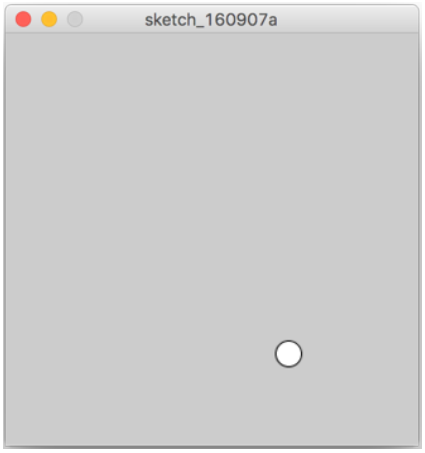
```
println(int(random(1, 7)));
```

13.3 サイコロをふって出た目の数を 5 つ表示



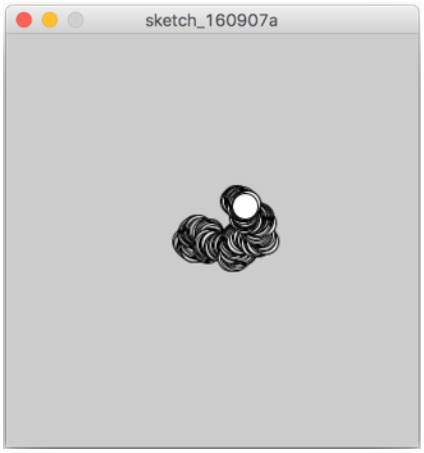

```
for (int i = 0; i < 5; i++) {
  println(int(random(1, 7)));
}
```

13.4 画面の中のランダムな位置に直径 20 の円を表示




```
size(320, 320);
ellipse(random(0, width), random(0, height), 20, 20);
```

13.5 画面の中央に直径 20 の円を表示。毎フレーム、その円をランダムに移動させる(上下左右に最大 2)。




```
float x, y;

void setup() {
  size(320, 320);
  x = width / 2;
  y = height / 2;
```

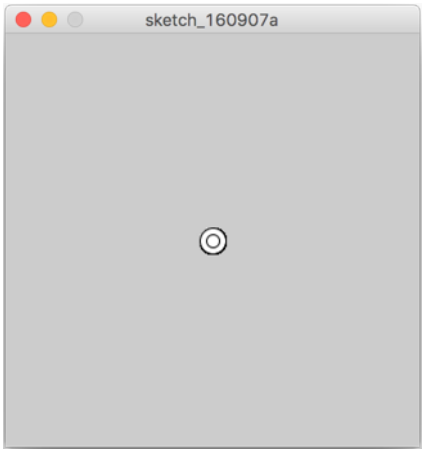
```
}

void draw() {
  x += random(-2, 2);
  y += random(-2, 2);
  ellipse(x, y, 20, 20);
}
```

14. 関数

14.1 説明

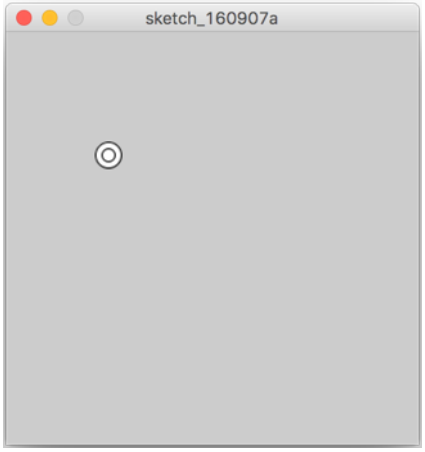
```
関数を使うには
1. 関数を宣言する
  戻り値の型 関数名(引数の型 引数名, ...) {
    実行させたい内容
    return 戻り値;
  }

2. 関数を呼び出す
  関数名(引数, ...);
```

14.2 画面の中央に◎を表示する関数 bullseye()を宣言し、呼出す。大きい円の直径 20、小さい円の直径 10 とする。



```
void bullseye() {
  ellipse(width / 2, height / 2, 20, 20);
  ellipse(width / 2, height / 2, 10, 10);
}

void setup() {
  size(320, 320);
}

void draw() {
  bullseye();
}
```

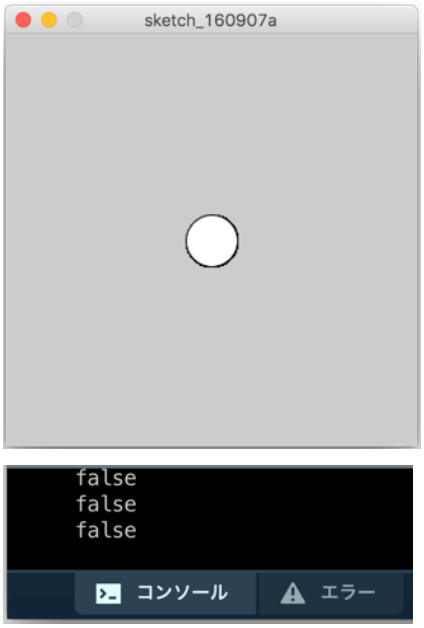
14.3 引数で指定した座標を中心に◎を表示する関数 bullseye()を宣言する。そして、引数としてマウスの座標を指定して呼出す。大きい円の直径 20、小さい円の直径 10 とする。残像は消すこと。




```
void bullseye(float x, float y) {
  ellipse(x, y, 20, 20);
  ellipse(x, y, 10, 10);
}
```

```
void setup() {
  size(320, 320);
}

void draw() {
  background(204);

  bullseye(mouseX, mouseY);
}
```

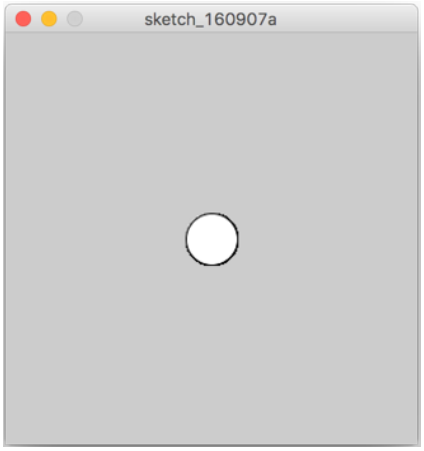
14.4 画面の中央に直径40の円を表示する。この円の中にマウスが入ると true、入っていないと false を返す関数 inside()を宣言する。inside()の戻り値をコンソールに表示




```
boolean inside() {
  boolean result; // 結果を入れる変数

  // 円の中にマウスが入ったら true、そうでなければ false
  if (dist(width / 2, height / 2, mouseX, mouseY) < 20) {
    result = true;
```

```
  } else {
    result = false;
  }

  // 結果を返す
  return result;
}

void setup() {
  size(320, 320);
}

void draw() {
  ellipse(width / 2, height / 2, 40, 40);
  println(inside());
}
```

14.5 画面の中央に直径40の円を表示する。この円の中にマウスが入るとtrue、入っていないと false を返す関数 inside()を宣言する。inside()の戻り値の値を利用して、マウスが円の中に入ると円を黒く、そうでなければ白く塗りつぶす




```
boolean inside() {
```

```
  boolean result; // 結果を入れる変数

  // 円の中にマウスが入ったら true、そうでなければ false
  if (dist(width / 2, height / 2, mouseX, mouseY) < 20) {
    result = true;
  } else {
    result = false;
  }

  // 結果を返す
  return result;
}

void setup() {
  size(320, 320);
}

void draw() {
  if (inside() == true) {
    fill(0);
  } else {
    fill(255);
  }
  ellipse(width / 2, height / 2, 40, 40);
}
```

15. オブジェクト

15.1 クラスとインスタンス

15.1.1 説明

オブジェクトとは、複数のデータとそれらに対する操作をまとめて、プログラムを部品として扱えるようにしたもの

Java では、オブジェクトは、クラスという設計図を書き、それからインスタンスという実体を作って使用する。インスタンスがオブジェクト。

オブジェクトを使うには

1. クラスを宣言する
2. クラスのインスタンスを表す変数を宣言する
3. クラスのインスタンスを生成し、変数に代入する
4. インスタンスを操作する

クラスの宣言

```
class クラス名 {
    フィールド
    コンストラクタ
    メソッド
}
```

フィールド: 変数の宣言と同様

メソッド: 関数の宣言と同様

コンストラクタ: インスタンスを生成するときに実行される関数のようなもの

クラスのインスタンスを表す変数の宣言

```
クラス名 変数名;
```

クラスのインスタンスを生成し、変数に代入

```
変数名 = new クラス名(引数);
```

インスタンスの操作

```
変数名.フィールド
変数名.メソッド()
```

15.1.2 ハンマーを表すクラス Hammer を宣言

```
class Hammer {
}
```

15.1.3 クラス Hammer に、画像 image、中心の座標 x、y をフィールドとして用意する

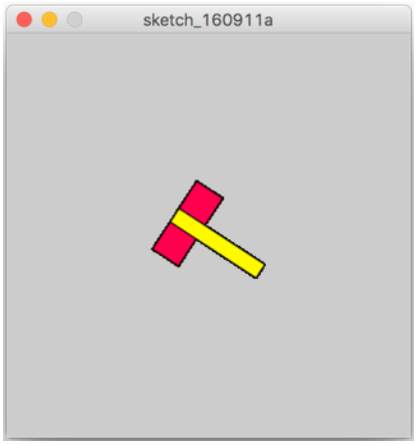
```
class Hammer {
  PImage image;
  float x;
  float y;
}
```

15.1.4 クラス Hammer に、画面に表示する display()メソッドを用意する

```
class Hammer {
  PImage image;
  float x;
  float y;

  void display() {
    image(image, x, y);
  }
}
```

15.1.5 クラス Hammer を使用するメイン・プログラムを作る。メイン・プログラムとクラス Hammer は別のタブにする。メイン・プログラムでは、インスタンスを生成し、画像 hammer.png と中心の座標を画面の中心に設定し、それを表示する。なお、imageMode(CENTER)を使い、

画像の中心の座標を指定して表示するモードにすること。



メイン・タブ

```
Hammer hammer;

void setup() {
  size(320, 320);
  imageMode(CENTER);

  hammer = new Hammer();
  hammer.image = loadImage("hammer.png");
  hammer.x = width / 2;
  hammer.y = height / 2;
}

void draw() {
  hammer.display();
}
```

Hammer タブ

```
class Hammer {
  PImage image;
  float x;
```

```
  float y;

  void display() {
    image(image, x, y);
  }
}
```

15.2 コンストラクタと this キーワード

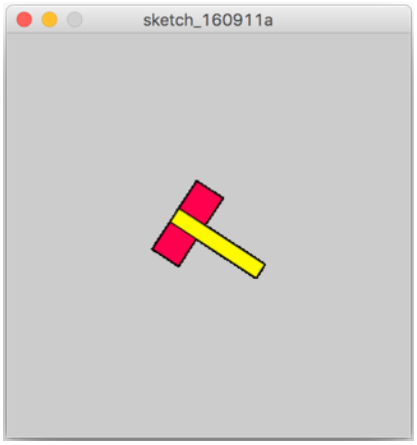
15.2.1 説明

```
コンストラクタとは、インスタンスを生成するときに実行される関数のようなもの
初期化をするのに使う
コンストラクタは、名前がクラス名で、戻り値のない関数として宣言する
  class クラス名 {
    // フィールドの宣言

    // コンストラクタの宣言
    クラス名(引数) {
      初期化の処理
    }

    // メソッドの宣言
  }
```

```
クラスの中で自分自身を表すには this キーワードを使用する
特に、コンストラクタの引数の名前とフィールドの名前が同じ場合に、フィールドの方に指定して区別するのに使われる
```

15.2.2 クラス Hammer にコンストラクタを用意し、それを利用してインスタンスを生成する。コンストラクタの引数は、中心の座標 tempX、tempY とする。また、コンストラクタの

中で image フィールドに loadImage("hammer.png")を代入して画像を初期化せよ。



メイン・タブ

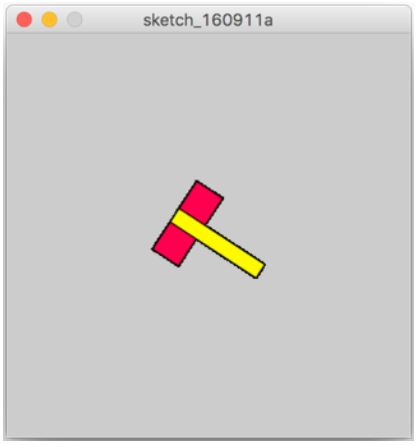
```
Hammer hammer;

void setup() {
  size(320, 320);
  imageMode(CENTER);

  hammer = new Hammer(width / 2, height / 2);
}

void draw() {
  hammer.display();
}
```

Hammer タブ

```
class Hammer {
  PImage image;
  float x;
  float y;

  Hammer(float tempX, float tempY) {
```

```
    image = loadImage("hammer.png");
    x = tempX;
    y = tempY;
  }

  void display() {
    image(image, x, y);
  }
}
```

15.2.3 クラス Hammer のコンストラクタの引数を tempX、tempY から x、y に変更し、this キーワードを利用して、値を設定する。また、Hammer クラスの宣言の中のフィールド image は、image()関数と紛らわしいので、this.image で統一する。



メイン・タブ

```
Hammer hammer;

void setup() {
  size(320, 320);
  imageMode(CENTER);

  hammer = new Hammer(width / 2, height / 2);
}
```

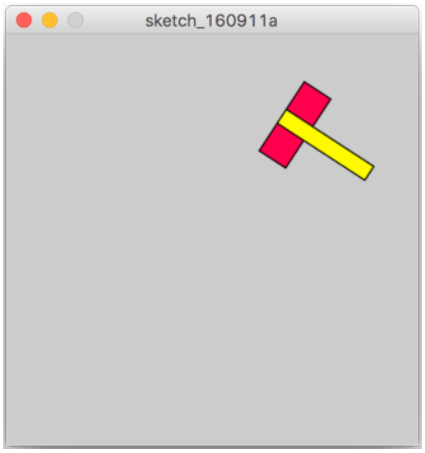
```
void draw() {
  hammer.display();
}
```

Hammer タブ

```
class Hammer {
  PImage image;
  float x;
  float y;

  Hammer(float x, float y) {
    this.image = loadImage("hammer.png");
    this.x = x;
    this.y = y;
  }

  void display() {
    image(this.image, x, y);
  }
}
```

15.2.4 クラス Hammer に、マウスの座標の位置に移動させるメソッド move()を用意する。

それを用いて、マウスの位置に画像を表示する。残像は消すこと。



メイン・タブ

```
Hammer hammer;

void setup() {
  size(320, 320);
  imageMode(CENTER);

  hammer = new Hammer(width / 2, height / 2);
}

void draw() {
  background(204);

  hammer.move();
  hammer.display();
}
```

Hammer タブ

```
class Hammer {
  PImage image;
  float x;
```

```
  float y;

  Hammer(float x, float y) {
    this.image = loadImage("hammer.png");
    this.x = x;
    this.y = y;
  }

  void display() {
    image(this.image, x, y);
  }

  void move() {
    x = mouseX;
    y = mouseY;
  }
}
```

15.3 オーバーロード

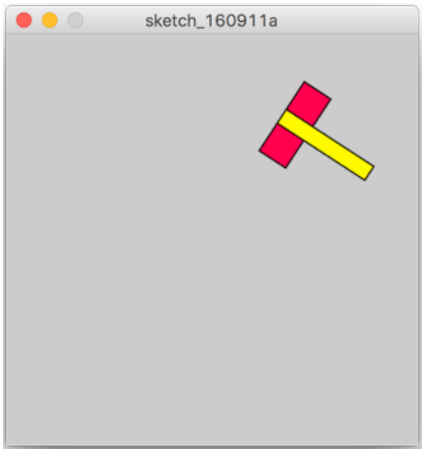
15.3.1 説明

オーバーロードとは、引数の個数や型を変えると、同じ名前でメソッドやコンストラクタを複数使えること
多重定義ともいう

引数のないコンストラクタをデフォルト・コンストラクタという
引数のあるコンストラクタを用意するとき、デフォルト・コンストラクタも用意した方が便利な場合がある

15.3.2 クラス Hammer にオーバーロードを用いて、コンストラクタを 2 つ用意する。1 つは座標 x、y の 2 つが引数で、それを初期値に設定する。もう 1 つは引数がなく、フィールド

x と y を画面の中心の座標に設定する。



メイン・タブ

```
Hammer hammer;

void setup() {
  size(320, 320);
  imageMode(CENTER);

  hammer = new Hammer(width / 2, height / 2);
}

void draw() {
  background(204);

  hammer.move();
  hammer.display();
}
```

Hammer タブ

```
class Hammer {
  PImage image;
  float x;
```

```
  float y;

  Hammer() {
    this.image = loadImage("hammer.png");
    this.x = width / 2;
    this.y = height / 2;
  }

  Hammer(float x, float y) {
    this.image = loadImage("hammer.png");
    this.x = x;
    this.y = y;
  }

  void display() {
    image(this.image, x, y);
  }

  void move() {
    x = mouseX;
    y = mouseY;
  }
}
```

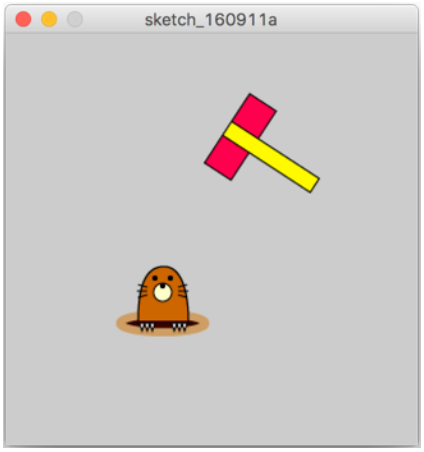
15.4 継承

15.4.1 説明

```
継承とは、クラス間で共通のプログラムを取り出して、使いまわせる機能
共通のプログラムを取り出したものを親クラスという
親クラスを継承した子クラスでは、親クラスに書いたプログラムがそのまま使える

  class 親クラス {
```

```
    親クラスのプログラム(子クラス 1 と 2 で共通の部分)を書く
  }

  class 子クラス 1 extends 親クラス {
    親クラスのプログラムは何もしなくても使える
    子クラス 1 の独自のプログラムを書く
  }

  class 子クラス 2 extends 親クラス {
    親クラスのプログラムは何もしなくても使える
    子クラス 2 の独自のプログラムを書く
  }
```

15.4.2 画像 mole.png をランダムな位置に表示する Mole クラスを追加する。フィールドは画像、x、y 座標。メソッドはランダムな位置への移動と表示が必要である。



メイン・タブ

```
Hammer hammer;
Mole mole;

void setup() {
  size(320, 320);
  imageMode(CENTER);
```

```
  hammer = new Hammer(width / 2, height / 2);
  mole = new Mole();
}

void draw() {
  background(204);

  hammer.move();
  mole.display();
  hammer.display();
}
```

Hammer タブ

```
class Hammer extends Sprite {
  PImage image;
  float x;
  float y;

  Hammer() {
    this.image = loadImage("hammer.png");
    this.x = width / 2;
    this.y = height / 2;
  }

  Hammer(float x, float y) {
    this.image = loadImage("hammer.png");
    this.x = x;
    this.y = y;
  }

  void display() {
```

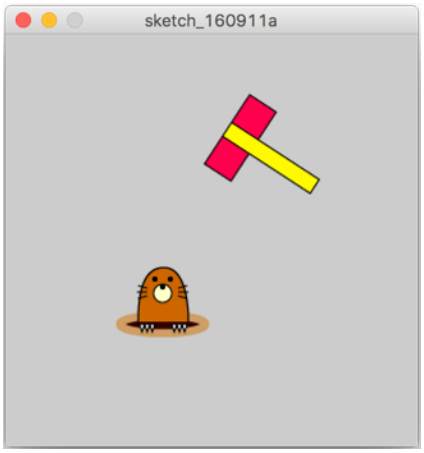
```
    image(this.image, x, y);
  }

  void move() {
    x = mouseX;
    y = mouseY;
  }
}
```

Mole タブ

```
class Mole extends Sprite {
  PImage image;
  float x;
  float y;

  Mole() {
    this.image = loadImage("mole.png");
    x = random(0, width);
    y = random(0, height);
  }

  void display() {
    image(this.image, x, y);
  }

  void move() {
    x = random(0, width);
    y = random(0, height);
  }
}
```

15.4.3 クラス Hammer と Mole には共通の部分がある。これを Sprite クラスとして取り出

し、継承するようにする。



メイン・タブ: 変更なし

Sprite タブ

```
class Sprite {
  PImage image;
  float x;
  float y;

  void display() {
    image(this.image, x, y);
  }
}
```

Hammer タブ

```
class Hammer extends Sprite {
  Hammer() {
    this.image = loadImage("hammer.png");
    this.x = width / 2;
    this.y = height / 2;
  }
```

```
  Hammer(float x, float y) {
    this.image = loadImage("hammer.png");
    this.x = x;
    this.y = y;
  }

  void move() {
    x = mouseX;
    y = mouseY;
  }
}
```

Mole タブ

```
class Mole extends Sprite {
  Mole() {
    this.image = loadImage("mole.png");
    x = random(0, width);
    y = random(0, height);
  }

  void move() {
    x = random(0, width);
    y = random(0, height);
  }
}
```

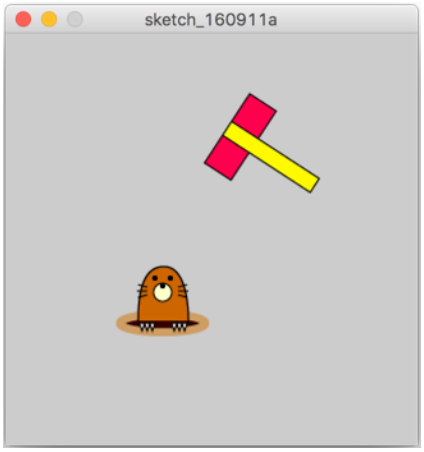
15.5 オーバーライドと super

15.5.1 説明

親クラスのコンストラクタやメソッドは、子クラスで同じ名前と同じ引数を用意すると、上書きすることができ、これをオーバーライドという。

子クラスでコンストラクタやメソッドをオーバーライドするときに、親クラスのコンストラクタやメソッドを呼び出したい場合があり、super を使用する。
単に super と書くと親クラスを表し、super(引数)と書くとコンストラクタが呼び出される。
子クラスのコンストラクタでは、super は必ず最初に記述する必要がある。

15.5.2 クラス Hammer と Mole のコンストラクタには、座標を設定している部分がある。この部分について、Sprite クラスにコンストラクタを用意して、それを呼び出すように変更する。Hammer と Mole のコンストラクタでは、super を必ず最初に記述する必要があることに注意する。



メイン・タブ: 変更なし

Sprite タブ

```
class Sprite {
  PImage image;
  float x;
  float y;

  Sprite() {
    x = width / 2;
    y = height / 2;
  }
```

```
  Sprite(float x, float y) {
    this.x = x;
    this.y = y;
  }

  void display() {
    image(this.image, x, y);
  }
}
```

Hammer タブ

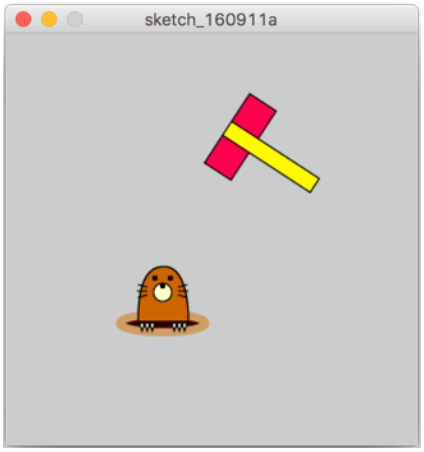
```
class Hammer extends Sprite {
  Hammer() {
    super(width / 2, height / 2);
    this.image = loadImage("hammer.png");
  }

  Hammer(float x, float y) {
    super(x, y);
    this.image = loadImage("hammer.png");
  }

  void move() {
    x = mouseX;
    y = mouseY;
  }
}
```

Mole タブ

```
class Mole extends Sprite {
  Mole() {
    super(random(0, width), random(0, height));
```

```
    this.image = loadImage("mole.png");
  }

  void move() {
    x = random(0, width);
    y = random(0, height);
  }
}
```

15.5.3 クラス Hammer と Mole にはどちらも move()メソッドがある。Sprite クラスにも空の move()メソッドを用意する。



メイン、Hammer、Mole タブ: 変更なし

Sprite タブ

```
class Sprite {
  PImage image;
  float x;
  float y;

  Sprite() {
    x = width / 2;
    y = height / 2;
```

```
  }

  Sprite(float x, float y) {
    this.x = x;
    this.y = y;
  }

  void display() {
    image(this.image, x, y);
  }

  void move() {
  }
}
```

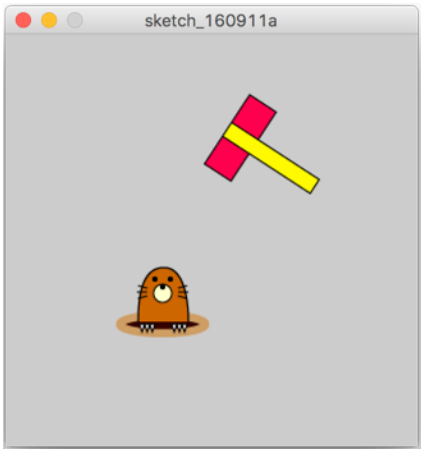
15.6 多態性

15.6.1 説明

子クラスは、親クラスの一種である(kind-of 関係)。
そのため、親クラスを表す変数に子クラスのインスタンスを代入できる。
これにより、共通の親クラスを用意すると、異なった子クラスのインスタンスを、共通のプログラムで処理できるようになる。

15.6.2 メイン・プログラムで、Hammer クラス、Mole クラスの変数を使用しているが、これ

らを Sprite クラスの変数に書き換えよ。



Sprite、Hammer、Mole タブ: 変更なし

メイン・タブ

```
Sprite hammer;
Sprite mole;

void setup() {
  size(320, 320);
  imageMode(CENTER);

  hammer = new Hammer(width / 2, height / 2);
  mole = new Mole();
}

void draw() {
  background(204);

  hammer.move();
  mole.display();
  hammer.display();
}
```

16. 配列

16.1 説明

配列とは、複数の同じ型のデータを扱えるしくみである。
※多態性を使うと、異なったクラスのインスタンスでも、共通の親クラスを持っていれば扱える。

配列を使用するには
1. 配列を表す変数を用意
   型[] 変数名;
2. 配列を生成して変数に代入
   変数名 = new 型[要素の個数];
   ※変数名[0]～変数名[要素の個数 - 1]までが使用できるようになる
3. 配列の要素を 1 つずつ代入
   変数名[添字] = 値;

配列の初期化: 配列に値を代入するところまでを一発で行う
   型[] 変数名 = { 要素 0, 要素 1, ... };

配列の要素の個数
配列は一種のオブジェクトであり、フィールドやメソッドを持っている。
配列の要素の個数は length フィールドに入っている。
   変数名.length

16.2 整数型で 10 個の要素を持つ配列 a を用意し、0～9 までの値を代入し、コンソールに表示する。

```
int[] a;
```

```
a = new int[10];
for (int i = 0; i < a.length; i++) {
  a[i] = i;
}
for (int i = 0; i < a.length; i++) {
  println(a[i]);
}
```

16.3 整数型で 10 個の要素を持つ配列 a を 0～9 までの値で初期化し、コンソールに表示する。

```
int[] a = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
for (int i = 0; i < a.length; i++) {
  println(a[i]);
}
```

16.4 小数型で 10 個の要素を持つ配列 a を用意し、100 未満の乱数を代入し、コンソールに表示する。

```
float[] a;
a = new float[10];
for (int i = 0; i < a.length; i++) {
  a[i] = random(100);
}
for (int i = 0; i < a.length; i++) {
  println(a[i]);
```

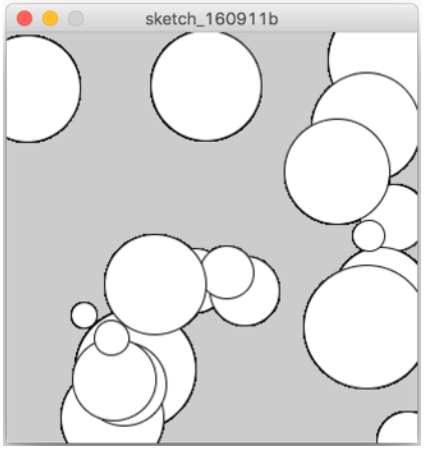
```
}
```

16.5 円を表すクラス Ball を作る。Ball のフィールドは座標 x、y、直径 d。コンストラクタでは、ランダムな座標を x、y に設定し、直径は 100 未満のランダムな値を設定する。また、表示するメソッド display()を用意する。

```
class Ball {
  float x;
  float y;
  float d;

  Ball() {
    x = random(width);
    y = random(height);
    d = random(100);
  }

  void display() {
    ellipse(x, y, d, d);
  }
}
```

16.6 クラス Ball のインスタンスを 20 個生成し、表示するメイン・プログラム

を作る。




```
Ball[] balls;

void setup() {
  size(320, 320);

  balls = new Ball[20];

  for (int i = 0; i < balls.length; i++) {
    balls[i] = new Ball();
  }
}

void draw() {
  for (int i = 0; i < balls.length; i++) {
    balls[i].display();
  }
}
```